

Lab 3: Transformation

I. Learning Goals

- Using ROS2
- Rigid Body transformations
- Tf2 library

II. Lab Policies

This is an individual assignment. You may discuss the assignment with other students; however, you must write and submit your own code. If you discuss the assignment with others, you are required to disclose the names of any students you worked with in the Canvas comments section when submitting your assignment.

Artificial Intelligence Disclosure:

If you used any generative AI tools to complete this assignment, you must disclose that you used them and how in the canvas comment section for the submission.

You may use generative AI in this course for brainstorming ideas, understanding concepts and documentation, debugging assistance, code explanation, refactoring suggestions, and improving clarity or organization of written or technical material.

You may NOT use generative AI to generate complete solutions, core project implementations, or competition code without substantial student contribution and understanding; during exams, in-class evaluations, or project demonstrations, unless explicitly permitted; or by uploading private assignment or project code to external AI services.

Please refer to the course syllabus for the complete policy on AI usage.

III. Overview

The goal of the lab is to show how coordinate transformations can be useful to find the location of objects relative to others. In the package provided, a ball will broadcast its pose moving 1m/s on a table. You will make 3 static frames on a robot on the table and 1 dynamic frame that is attached to the ball. You will then create a listener node that computes the ball's position relative to the base of the robot.

Static Broadcasters:

You will create 3 static broadcasters using the following command:

Shell

```
ros2 run tf2_ros static_transform_publisher [x] [y] [z] [yaw]  
[pitch] [roll] [frame_id] [child_frame_id]
```

In 3 separate terminals do the following:

Create a frame called base with frame_id world at (0,0,0), a frame called end_effector_joint with frame_id base at (34, 34, 41), and finally create a frame called camera with frame_id world at (34, 23, 60) with yaw pitch roll of (0,0,-1).

Dynamic Broadcaster:

You will create a dynamic broadcaster that broadcasts the ball's reference frame. Complete the skeleton code provided in ball_attached_frame.py. Add an arbitrary offset from the frame of the ball and the ball's pose. The [tf2 reference](#) might help.

Listener Node:

Using the skeleton code provided, complete the listener node. The Target frame should be the base and the source frame should be the camera. You print to the console the coordinates of the ball relative to the base of the robot.

IV. Visualizing Frames

The following command is how you can see the translation and rotation for each frame:

Shell

```
ros2 run tf2_ros tf2_echo [source_frame][target_frame]
```

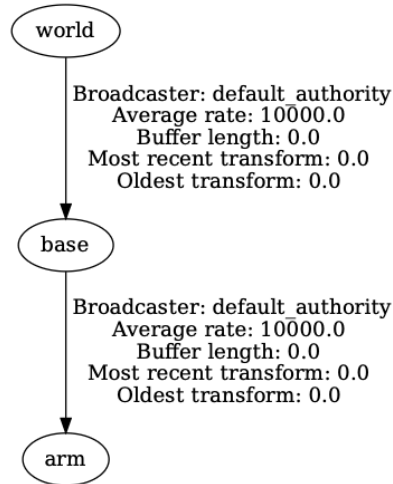
Additionally, you can generate a pdf file displaying a graph of how the frames interact with each other. Use the following command to do so

Shell

```
ros2 run tf2_tools view_frames.py
```

If you are working in a docker container the pdf file should save to workspace, and you can view it locally outside the docker container.

view_frames Result
Recorded at time: 1770309854.763911



VI: Deliverables and Submission

You can implement this package in either Python or C++, and skeleton code for the Python version has been provided in the zip file attached to this assignment page. Use the docker container and workspace you setup for lab 1, or alternatively create a new docker container and workspace.

Deliverable: To submit your work, upload a video to the Canvas page with a short video of the console running the listener node. Additionally, upload the pdf of the graph produced by the `tf2_tools` command. A portion of the submissions will be peer reviewed by course staff to ensure that the code is academically honest(i.e. video is not faked, code implements transforms).